

If you have multiple jobs on a Jenkins dashboard and want to filter and organize them, you can use the **Views** option. In the Jenkins console, under **View**, you can create specific views for different types of jobs. For instance, you can create views for the development (DevOps) team and the operations team.

Setting Up Upstream and Downstream Jobs

If you need to run multiple jobs in sequence, you can use Jenkins' **Upstream** and **Downstream** features.

1. Go to the job configuration by clicking **Configure**.
2. Scroll to the **Post-build Actions** section.
3. Select **Build other projects** and choose the next job you want to trigger.
4. Save the configuration.

Now, when you trigger **Job 1**, the subsequent jobs will automatically execute in sequence. The remaining jobs will stay in the build queue until **Job 1** completes. This method of linking jobs through **Upstream** and **Downstream** works like a pipeline.

Visualizing Pipelines with Plugins

To represent pipelines visually, you can install plugins like the **Build Pipeline** and **Delivery Pipeline** plugins.

1. Once the plugins are installed, go to the **Views** option.
 2. Create a new view by giving it a name, configure it, and save it.
 3. The Jenkins pipeline will then be displayed in a visual diagram format.
-

Pipeline as Code

For complex pipelines, it is recommended to use **Pipeline as Code** via Groovy scripts. Although Jenkins has a GUI to configure pipelines, using Groovy scripts offers more flexibility.

In the **Jenkinsfile**, you can define **stages** like **Clone**, **Build**, and **Deploy**. The pipeline will follow these stages to execute.

If you're a beginner, use the **Pipeline Syntax Helper** in Jenkins to generate code. Usually, a pipeline has at least three stages: **Build**, **Test**, and **Deploy**. Writing the script logically is important to ensure success.

Storing and Executing Scripts

Once your Groovy pipeline script is ready, store it in your **GitHub** repository for team review and collaboration.

To execute the pipeline script from GitHub, configure Jenkins to use **Pipeline SCM**. This allows Jenkins to pull the pipeline code directly from the GitHub repository and run it.

Example Jenkins Pipeline Script

Here's a basic Jenkins pipeline script using Groovy:

```
pipeline {
    agent any

    stages {
        stage('Clone') {
            steps {
                // Clone your GitHub repository
                git 'https://github.com/your-repo/your-project.git'
            }
        }

        stage('Build') {
            steps {
                // Run build commands, e.g., for Maven projects
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            steps {
                // Run tests, for example, with Maven
                sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                // Example deployment step
                echo 'Deploying to production...'
                // You can add deployment scripts or commands here
            }
        }
    }
}
```